Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next item →

Build an AI text generator

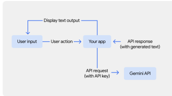
Overview

In this activity, you'll build a simple but powerful AI Text Generator application using the Gemini API. You'll write code that takes a text instruction from a user, sends it to the Gemini API, and displays the AI-generated response. This project helps you practice the key skills of connecting to an API, sending requests, and handling the response and errors. These are the key skills for creating AI applications.

Scenario

You are a new developer at an educational technology company tasked with building a tool to help students with their homework. The first task is to build a prototype. This prototype will be a simple application where a student can type a question, like "Explain photosynthesis in a simple paragraph," and the tool will use the Gemini API to generate an accurate answer.

General text generator diagram



Prerequisites

Before starting this project, make sure you have:

1. **Python or Node.js/JavaScript** installed on your computer.
2. **A Gemini API key.** You can get a free key quickly from [Google AI Studio].

- *An **API key** is a unique code that grants access to the API. It must be kept secret, like a password.*

Steps

Step 1: Choose your programming language

Decide if you will use Python or JavaScript for this project. The steps are similar for both, but the code and commands will be slightly different.

- **Tip:** Many people find Python's syntax to be very clear and easy to read when they are new to programming.
- **Common mistake:** Overlooking the separate installation commands for each language's SDK (Software Development Kit). Make sure to use the correct one in the next step.

1. **You are setting up your environment to build an AI application. You decide to use the JavaScript SDK but accidentally try to install it using the Python package manager command (`pip install`). What is the most likely result of this action? Select the best answer.** 1 / 1 point

- (●) The command will fail because package managers like `pip` and `npm` are specific to their respective programming languages.
- ( ) The computer will automatically switch the developer's code to Python to match the command.
- ( ) The application will run, but the AI will only be able to understand Python-related questions.
- ( ) The installation will work perfectly because all programming languages use the same libraries.

✓ Nice work

Correct! Tools like `pip` (Python) and `npm` (JavaScript) are designed to fetch libraries specifically for their own language. You must use the tool that matches your code.

Step 2: Set up your project and install the SDK

First, create a new folder for your project to keep your files organized. Then, open your terminal or command prompt, go into the new folder, and install the official Google SDK. The SDK is a library that makes it much easier to use the Gemini API in your code.

- For **Python**, run this command: `pip install google-genai`
- For **JavaScript/Node.js**, run this command: npm install @google/genai

Common mistake: Running the installation command outside of the project folder. Ensure the command is run in the correct folder.

2. **A developer wants to use the Gemini API in their Python application. They could manually make HTTP requests, or they could use the official Google Generative AI SDK. What is the primary benefit of using the Software Development Kit (SDK) instead of making manual requests? Select the best answer.** 1 / 1 point

- ( ) The SDK makes the application run faster than a manual request.
- ( ) The SDK is the only way to get an API key.
- ( ) The SDK automatically designs a user interface for the application.
- (●) The SDK provides pre-built functions that make it much simpler to send requests and handle responses.

✓ Nice work

Correct! The SDK acts as a helper library. It simplifies complex tasks like authentication, request formatting, and response parsing into easy-to-use functions.

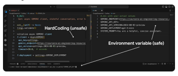
Step 3: Secure your API key

The API key must never be written directly in your code. This is a security risk. Instead, use an environment variable. An environment variable is a value saved on your computer, outside the project, that your code can access securely.

1. Set up an environment variable named GEMINI_API_KEY and set its value to the key from Google AI Studio. (The method for doing this depends on the operating system).
2. Your code will read this variable, meaning the key does not have to be put in your code file.

Common mistake: Writing the API key directly into the script. This is called hardcoding and is not secure.

Hardcoding vs. environment variable



3. **You are building the AI Text Generator from this project. You need to include your Gemini API key in the code but want to share your project on a public code-hosting website later. What is the most secure way to manage the API key in the project? Select the best answer.** 1 / 1 point

- ( ) Write the key directly in the main code file inside quotation marks.
- ( ) Email the key to a coworker for safekeeping and ask for it when needed.
- ( ) Save the key in a text file named `keys.txt` and include it with the project files.
- (●) Store the key in an environment variable on your computer and have the code read it from there.

✓ Nice work

Correct! Using an environment variable keeps the key separate from the code. This prevents it from being accidentally exposed when the code is shared.

Step 4: Write the code to call the API

Your application must perform three core tasks:

1. Ask the user to enter a text prompt.
2. Send that prompt to the Gemini API.
3. Print the text part of the API's response.

Tip: Start by creating a function that contains the API call logic. This keeps your code clean and easy to read. Make sure to have an environment variable for your API key.

Common mistake: Printing the entire `response` object instead of just the generated text. Always remember to access the `.text` property (e.g., `response.text`) to see the final output, or your console will be filled with unnecessary metadata.

4. **You have written your code but notice that the output in the terminal includes a lot of extra information like "finish_reason," "index," and "safety_ratings" instead of just the answer to the prompt. What is the most likely cause of this? Select the best answer.** 1 / 1 point

- ( ) You forgot to connect to the internet before running the code.
- (●) You are printing the entire response object instead of accessing the specific .text property.
- ( ) The prompt entered by the user was too short for the AI to understand.
- ( ) The Gemini API is broken and sending the wrong information.

✓ Nice work

Correct! The API response contains a lot of "metadata" (extra details). To show only the generated answer, you must specifically tell the code to print the text property.

Step 5: Add error handling

If the internet is down or the API key is wrong, your application might crash. To prevent this, add error handling. Use a `try...except` block in Python or a `try...catch` block in JavaScript. This tells the program to try running the API call, and if it fails, catch the error and print a helpful message instead of crashing.

Tip: The error message should be simple and clear for the user, such as, "Sorry, an error occurred. Please check your API key and internet connection."

5. **While testing the AI Text Generator, a user enters an invalid API key. The application crashes and shows a long, confusing error message to the user. What programming concept should be used to catch this API error and display a friendly message instead of crashing? Select the best answer.** 1 / 1 point

- ( ) A function to organize the API call code
- (●) A `try...except` block (or `try...catch` in JavaScript) to handle potential errors gracefully
- ( ) An `if/else` statement to check the user's input length
- ( ) A `for` loop to retry the connection multiple times

✓ Nice work

Correct! This structure allows the program to "try" a piece of code that might fail. If an error occurs, the "except" or "catch" block runs, preventing a crash and allowing the developer to show a helpful message.

Step 6: Test your application

Run the application from the terminal. Test it with different prompts to see what it generates.

- **Example prompts:**
  - "Write a short poem about the moon."
  - "What are three fun facts about the ocean?"
  - "Give me a recipe idea for dinner with chicken and rice."

Success checklist

Your project is complete when:

- Your application successfully gets a response from the Gemini API.
- The generated text is clearly displayed to the user.
- The application does not crash if an API error occurs.
- Your API key is loaded securely from an environment variable.

Key points

- API Integration requires precise authentication and secure key management. Never hardcode API keys directly into your source code.
- The SDK (Software Development Kit) simplifies complex tasks, making it much easier to connect and communicate with the Gemini API.
- **Error handling** (using try...except or try...catch blocks) enhances application reliability by preventing crashes and providing clear messages to users.

Exemplar solution

Here is a sample solution in Python. Compare it to what you have built.

```python
import os
from google import genai
from dotenv import load_dotenv

def main():
    # Load environment variables from .env file
    load_dotenv()

    # Get the API key from the environment
    api_key = os.getenv("GEMINI_API_KEY")

    if not api_key:
        print("Error: GEMINI_API_KEY not found.")
        print("Please create a .env file and add your API key.")
        return

    try:
        # Create the client (google-genai SDK)
        client = genai.Client(api_key=api_key)

        response = client.models.generate_content(
            model="gemini-3-flash-preview",
            contents="Write a short poem about the moon."
        )

        # Print the response
        print("\n--- AI Response ---")
        print(response.text)
        print("-------------------")

    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    main()
```

How does your work compare?

- Does your application load the API key from an environment variable?
- Does your error handling provide a clear message if something goes wrong?
- Is the output formatted in a way that is easy for the user to read?

Next steps

Build upon this application by integrating advanced features like model selection or structured output, which are covered in later modules.